

```
import pandas as pd
import matplotlib.pyplot as plt

# Ustawienia dla polskich znaków
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
```

Wstęp do pandas

Pandas to biblioteka w Pythonie, która umożliwia łatwe i efektywne manipulowanie oraz analizowanie danych.

Jest szczególnie przydatna do pracy z danymi tabelarycznymi, podobnymi do tych w arkuszach kalkulacyjnych.

CZĘŚĆ 1: Tworzenie DataFrame z polskimi danymi

```
# Przykład 1: Dane o polskich miastach
data_miasta = {
    'Miasto': ['Warszawa', 'Kraków', 'Łódź', 'Wrocław', 'Poznań',
               'Gdańsk', 'Szczecin'],
    'Województwo': ['mazowieckie', 'małopolskie', 'łódzkie',
                    'dolnośląskie',
                    'wielkopolskie', 'pomorskie',
                    'zachodniopomorskie'],
    'Populacja': [1863056, 779966, 672185, 643782, 532048, 470907,
                  395513],
    'Powierzchnia_km2': [517, 327, 293, 293, 262, 262, 301],
    'Rok_nadania_praw': [1413, 1257, 1423, 1242, 1253, 1263, 1243]
}
```

```
miasta = pd.DataFrame(data_miasta)
print(miasta)
```

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				
2	Łódź	łódzkie	672185	293
1423				
3	Wrocław	dolnośląskie	643782	293
1242				
4	Poznań	wielkopolskie	532048	262
1253				
5	Gdańsk	pomorskie	470907	262
1263				

6	Szczecin	zachodniopomorskie	395513	301
1243				

CZĘŚĆ 2: PODSTAWOWE METODY INSPEKCJI DANYCH

```
print(">>> .head() - Pierwsze 5 wierszy (lub inna liczba)")
print(miasta.head(3))

>>> .head() - Pierwsze 5 wierszy (lub inna liczba)
      Miasto Województwo Populacja Powierzchnia_km2
Rok_nadania_praw
0  Warszawa mazowieckie   1863056             517
1413
1   Kraków  małopolskie    779966             327
1257
2     Łódź    łódzkie     672185             293
1423

print(">>> .info() - Informacje o strukturze danych")
miasta.info()

>>> .info() - Informacje o strukturze danych
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Miasto                7 non-null    object
1   Województwo           7 non-null    object
2   Populacja             7 non-null    int64
3   Powierzchnia_km2      7 non-null    int64
4   Rok_nadania_praw      7 non-null    int64
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes

print(">>> .shape - Wymiary DataFrame (wiersze, kolumny)")
print(f"Nasz DataFrame ma: {miasta.shape[0]} wierszy i
{miasta.shape[1]} kolumn")

>>> .shape - Wymiary DataFrame (wiersze, kolumny)
Nasz DataFrame ma: 7 wierszy i 5 kolumn

print(">>> .describe() - Statystyki opisowe dla kolumn numerycznych")
print(miasta.describe())

>>> .describe() - Statystyki opisowe dla kolumn numerycznych
      Populacja  Powierzchnia_km2  Rok_nadania_praw
count  7.000000e+00             7.000000          7.000000
mean   7.653510e+05            322.142857          1299.142857
std    5.010882e+05            88.856492           81.581102
min    3.955130e+05            262.000000          1242.000000
```

25%	5.014775e+05	277.500000	1248.000000
50%	6.437820e+05	293.000000	1257.000000
75%	7.260755e+05	314.000000	1338.000000
max	1.863056e+06	517.000000	1423.000000

```
print(">>> .columns - Nazwy kolumn")
```

```
print(miasta.columns.tolist())
```

```
>>> .columns - Nazwy kolumn
```

```
['Miasto', 'Województwo', 'Populacja', 'Powierzchnia_km2',  
'Rok_nadania_praw']
```

```
print(">>> .index - Indeks wierszy")
```

```
print(miasta.index)
```

```
>>> .index - Indeks wierszy
```

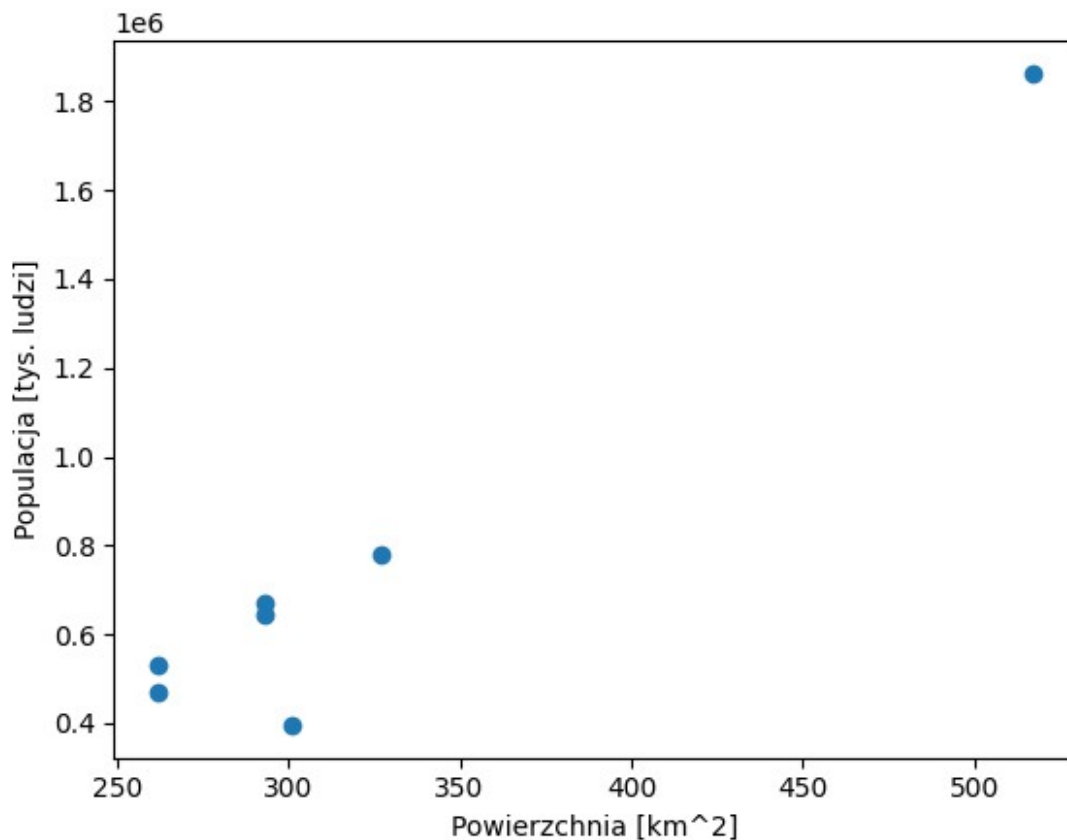
```
RangeIndex(start=0, stop=7, step=1)
```

```
plt.scatter(miasta["Powierzchnia_km2"], miasta["Populacja"])
```

```
plt.xlabel("Powierzchnia [km^2]")
```

```
plt.ylabel("Populacja [tys. ludzi]")
```

```
plt.show()
```



CZĘŚĆ 3: SORTOWANIE DANYCH

```
print("1. Sortowanie rosnąco po populacji:")
print(miasta.sort_values("Populacja").head())
```

1. Sortowanie rosnąco po populacji:

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
6	Szczecin	zachodniopomorskie	395513	301
1243				
5	Gdańsk	pomorskie	470907	262
1263				
4	Poznań	wielkopolskie	532048	262
1253				
3	Wrocław	dolnośląskie	643782	293
1242				
2	Łódź	łódzkie	672185	293
1423				

```
print("2. Sortowanie malejąco po populacji:")
print(miasta.sort_values("Populacja", ascending=False).head())
```

2. Sortowanie malejąco po populacji:

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				
2	Łódź	łódzkie	672185	293
1423				
3	Wrocław	dolnośląskie	643782	293
1242				
4	Poznań	wielkopolskie	532048	262
1253				

```
print("3. Sortowanie według wielu kolumn:")
print("    (Województwo rosnąco, potem Populacja malejąco)")
print(miasta.sort_values(["Województwo", "Populacja"],
                          ascending=[True, False]))
```

3. Sortowanie według wielu kolumn:
(Województwo rosnąco, potem Populacja malejąco)

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
3	Wrocław	dolnośląskie	643782	293
1242				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				

5	Gdańsk	pomorskie	470907	262
1263				
4	Poznań	wielkopolskie	532048	262
1253				
6	Szczecin	zachodniopomorskie	395513	301
1243				
2	Łódź	łódzkie	672185	293
1423				

CZĘŚĆ 4: WYBIERANIE KOLUMN

```
print("1. Jedna kolumna (zwraca Series):")
print(miasta["Miasto"])
print(f"\nTyp: {type(miasta['Miasto'])}")
```

1. Jedna kolumna (zwraca Series):

```
0    Warszawa
1     Kraków
2      Łódź
3    Wrocław
4     Poznań
5     Gdańsk
6    Szczecin
```

Name: Miasto, dtype: object

Typ: <class 'pandas.core.series.Series'>

```
print("2. Wiele kolumn (zwraca DataFrame):")
print(miasta[["Miasto", "Populacja"]])
```

2. Wiele kolumn (zwraca DataFrame):

```
   Miasto  Populacja
0  Warszawa   1863056
1   Kraków    779966
2    Łódź     672185
3  Wrocław   643782
4   Poznań   532048
5   Gdańsk   470907
6  Szczecin   395513
```

```
print("3. Używanie zmiennej z listą kolumn:")
kolumny_do_wyswietlenia = ["Miasto",
                           "Województwo",
                           "Populacja"]
print(miasta[kolumny_do_wyswietlenia])
```

3. Używanie zmiennej z listą kolumn:

```
   Miasto  Województwo  Populacja
0  Warszawa   mazowieckie   1863056
1   Kraków   małopolskie    779966
```

2	Łódź	łódzkie	672185
3	Wrocław	dolnośląskie	643782
4	Poznań	wielkopolskie	532048
5	Gdańsk	pomorskie	470907
6	Szczecin	zachodniopomorskie	395513

CZĘŚĆ 5: FILTROWANIE WIERSZY

```
print("1. Miasta z populacją powyżej 600 000:")
print(miasta[miasta["Populacja"] > 600000])
```

```
1. Miasta z populacją powyżej 600 000:
      Miasto Województwo Populacja Powierzchnia_km2
Rok_nadania_praw
0  Warszawa mazowieckie  1863056          517
1413
1   Kraków  małopolskie   779966          327
1257
2     Łódź   łódzkie     672185          293
1423
3  Wrocław dolnośląskie  643782          293
1242
```

```
print("2. Miasta z województwa mazowieckiego:")
print(miasta[miasta["Województwo"] == "mazowieckie"])
```

```
2. Miasta z województwa mazowieckiego:
      Miasto Województwo Populacja Powierzchnia_km2
Rok_nadania_praw
0  Warszawa mazowieckie  1863056          517
1413
```

```
print("3. Miasta z prawami miejskimi przed 1300 rokiem:")
print(miasta[miasta["Rok_nadania_praw"] < 1300])
```

```
3. Miasta z prawami miejskimi przed 1300 rokiem:
      Miasto Województwo Populacja Powierzchnia_km2
Rok_nadania_praw
1   Kraków  małopolskie   779966          327
1257
3  Wrocław dolnośląskie  643782          293
1242
4  Poznań  wielkopolskie  532048          262
1253
5   Gdańsk  pomorskie    470907          262
1263
6  Szczecin zachodniopomorskie 395513          301
1243
```

```
print("4. Złożone warunki z operatorem AND (&):")
print("    Miasta powyżej 500k ludności I powierzchnia powyżej 280 km²")
```

```
duze_miasta = miasta[(miasta["Populacja"] > 500000) &
                      (miasta["Powierzchnia_km2"] > 280)]
print(duze_miasta)
```

4. Złożone warunki z operatorem AND (&):

Miasta powyżej 500k ludności I powierzchnia powyżej 280 km²

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				
2	Łódź	łódzkie	672185	293
1423				
3	Wrocław	dolnośląskie	643782	293
1242				

```
print("5. Złożone warunki z operatorem OR (|):")
```

```
print("    Miasta z Warszawy LUB Krakowa")
```

```
stolice = miasta[(miasta["Miasto"] == "Warszawa") |
                  (miasta["Miasto"] == "Kraków")]
print(stolice)
```

5. Złożone warunki z operatorem OR (|):

Miasta z Warszawy LUB Krakowa

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				

```
print("6. Używanie .isin() dla wielu wartości:")
```

```
wybrane_miasta = ["Warszawa", "Kraków", "Wrocław"]
```

```
print(miasta[miasta["Miasto"].isin(wybrane_miasta)])
```

6. Używanie .isin() dla wielu wartości:

	Miasto	Województwo	Populacja	Powierzchnia_km2
Rok_nadania_praw				
0	Warszawa	mazowieckie	1863056	517
1413				
1	Kraków	małopolskie	779966	327
1257				
3	Wrocław	dolnośląskie	643782	293
1242				

CZĘŚĆ 6: DODAWANIE NOWYCH KOLUMN

```
print("1. Obliczanie gęstości zaludnienia:")
miasta["Gęstość_na_km2"] = miasta["Populacja"] /
miasta["Powierzchnia_km2"]
print(miasta[["Miasto", "Populacja", "Powierzchnia_km2",
"Gęstość_na_km2"]])
```

1. Obliczanie gęstości zaludnienia:

	Miasto	Populacja	Powierzchnia_km2	Gęstość_na_km2
0	Warszawa	1863056	517	3603.589942
1	Kraków	779966	327	2385.217125
2	Łódź	672185	293	2294.146758
3	Wrocław	643782	293	2197.208191
4	Poznań	532048	262	2030.717557
5	Gdańsk	470907	262	1797.354962
6	Szczecin	395513	301	1313.996678

```
print("2. Kategoryzacja miast według wielkości:")
```

```
def kategoryzuj_miasto(populacja):
    if populacja > 1000000:
        return "Bardzo duże"
    elif populacja > 500000:
        return "Duże"
    else:
        return "Średnie"
```

```
miasta["Kategoria"] = miasta["Populacja"].apply(kategoryzuj_miasto)
print(miasta[["Miasto", "Populacja", "Kategoria"]])
```

2. Kategoryzacja miast według wielkości:

	Miasto	Populacja	Kategoria
0	Warszawa	1863056	Bardzo duże
1	Kraków	779966	Duże
2	Łódź	672185	Duże
3	Wrocław	643782	Duże
4	Poznań	532048	Duże
5	Gdańsk	470907	Średnie
6	Szczecin	395513	Średnie

Zadania

```
# Dataset do zadań
temperatura = pd.DataFrame({
    'Miasto': ['Warszawa', 'Kraków', 'Gdańsk', 'Wrocław', 'Poznań',
'Łódź'],
    'Styczeń': [-2, -1, 0, -1, -2, -3],
    'Kwiecień': [9, 10, 8, 11, 10, 9],
    'Lipiec': [19, 20, 17, 21, 19, 20],
```

```

    'Październik': [9, 9, 10, 10, 9, 8]
})

print("Dataset 'temperatura' - średnie temperatury w polskich miastach:")
print(temperatura)

```

Dataset 'temperatura' - średnie temperatury w polskich miastach:

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
0	Warszawa	-2	9	19	9
1	Kraków	-1	10	20	9
2	Gdańsk	0	8	17	10
3	Wrocław	-1	11	21	10
4	Poznań	-2	10	19	9
5	Łódź	-3	9	20	8

Zadanie 1: podstawowa inspekcja

Używając DataFrame `temperatura`, wykonaj następujące operacje:

- Wyświetl pierwsze 3 wiersze
- Wyświetl informacje o typach danych (`.info()`)
- Wyświetl kształt DataFrame (`.shape`)
- Wyświetl tylko nazwy kolumn (`.columns`)

```

print("Pierwsze 3 wiersze:")
print(temperatura.head(3))

```

Pierwsze 3 wiersze:

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
0	Warszawa	-2	9	19	9
1	Kraków	-1	10	20	9
2	Gdańsk	0	8	17	10

```

print("Informacje o typach danych")
temperatura.info()

```

Informacje o typach danych

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Miasto          6 non-null     object
1   Styczeń         6 non-null     int64
2   Kwiecień        6 non-null     int64
3   Lipiec          6 non-null     int64
4   Październik     6 non-null     int64

```

```

dtypes: int64(4), object(1)
memory usage: 372.0+ bytes

print(f"Nasz DataFrame ma: {temperatura.shape[0]} wierszy i {temperatura.shape[1]} kolumn")

Nasz DataFrame ma: 6 wierszy i 5 kolumn

print("Nazwy kolumn:")
print(temperatura.columns.tolist())

Nazwy kolumn:
['Miasto', 'Styczeń', 'Kwiecień', 'Lipiec', 'Październik']

```

Zadanie 2: sortowanie

- Posortuj miasta według temperatury w lipcu (rosnąco)
- Posortuj miasta według temperatury w styczniu (malejąco)
- Posortuj miasta według temperatury w lipcu (malejąco), a następnie według kwietnia (rosnąco)

```

print("a) Sortowanie rosnąco po temperaturze w lipcu:")
print(temperatura.sort_values(by="Lipiec", ascending=True))

print("\nb) Sortowanie malejąco po temperaturze w styczniu:")
print(temperatura.sort_values(by="Styczeń", ascending=False))

print("\nc) Sortowanie malejąco po lipcu, a następnie rosnąco po kwietniu:")
print(temperatura.sort_values(by=["Lipiec", "Kwiecień"], ascending=[False, True]))

```

a) Sortowanie rosnąco po temperaturze w lipcu:

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
2	Gdańsk	0	8	17	10
0	Warszawa	-2	9	19	9
4	Poznań	-2	10	19	9
1	Kraków	-1	10	20	9
5	Łódź	-3	9	20	8
3	Wrocław	-1	11	21	10

b) Sortowanie malejąco po temperaturze w styczniu:

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
2	Gdańsk	0	8	17	10
1	Kraków	-1	10	20	9
3	Wrocław	-1	11	21	10
0	Warszawa	-2	9	19	9
4	Poznań	-2	10	19	9

5	Łódź	-3	9	20	8
---	------	----	---	----	---

c) Sortowanie malejąco po lipcu, a następnie rosnąco po kwietniu:

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
3	Wrocław	-1	11	21	10
5	Łódź	-3	9	20	8
1	Kraków	-1	10	20	9
0	Warszawa	-2	9	19	9
4	Poznań	-2	10	19	9
2	Gdańsk	0	8	17	10

Zadanie 3: Wybieranie kolumn

a) Wybierz tylko kolumnę 'Miasto' (wynik: Series)

b) Wybierz kolumny 'Miasto' i 'Lipiec' (wynik: DataFrame)

c) Wybierz kolumny 'Miasto', 'Styczeń' i 'Lipiec', używając zmiennej z listą nazw kolumn

```
print("a) Tylko kolumna Miasto:")
print(temperatura["Miasto"])
print(f"Typ: {type(temperatura['Miasto'])}")

print("\nb) Tylko kolumny Miasto i Lipiec:")
print(temperatura[["Miasto", "Lipiec"]])
print(f"Typ: {type(temperatura[['Miasto', 'Lipiec']])}")

print("\nc) Wybór kolumny Miasto, Styczeń i Lipiec, używając zmiennej z listą nazw kolumn:")
kolumny_do_wyswietlenia = ["Miasto", "Styczeń", "Lipiec"]
print(temperatura[kolumny_do_wyswietlenia])
```

a) Tylko kolumna Miasto:

```
0    Warszawa
1     Kraków
2     Gdańsk
3     Wrocław
4     Poznań
5     Łódź
```

Name: Miasto, dtype: object

Typ: <class 'pandas.core.series.Series'>

b) Tylko kolumny Miasto i Lipiec:

```
   Miasto  Lipiec
0  Warszawa    19
1   Kraków    20
2   Gdańsk    17
3  Wrocław    21
4   Poznań    19
5    Łódź     20
```

Typ: <class 'pandas.core.frame.DataFrame'>

c) Wybór kolumny Miasto, Styczeń i Lipiec, używając zmiennej z listą nazw kolumn:

	Miasto	Styczeń	Lipiec
0	Warszawa	-2	19
1	Kraków	-1	20
2	Gdańsk	0	17
3	Wrocław	-1	21
4	Poznań	-2	19
5	Łódź	-3	20

Zadanie 4: Filtrowanie

a) Znajdź miasta, gdzie temperatura w lipcu jest większa niż 19 stopni

b) Znajdź miasta, gdzie temperatura w styczniu jest większa lub równa -1

c) Znajdź miasta, gdzie temperatura w kwietniu wynosi dokładnie 10 stopni

d) Znajdź miasta, gdzie temperatura w lipcu jest 20 LUB temperatura w styczniu jest -1

```
print("a) Miasta gdzie temperatura w lipcu większa niż 19 stopni")
print(temperatura[temperatura["Lipiec"] > 19])

print("\n b) Miasta gdzie temperatura w styczniu większa lub równa -1")
print(temperatura[temperatura["Styczeń"] >= -1])

print("\n c) Miasta gdzie temperatura w kwietniu wynosi dokładnie 10 stopni")
print(temperatura[temperatura["Kwiecień"] == 10])

print("\n d) Miasta gdzie temperatura w lipcu jest 20 LUB temperatura w styczniu jest -1")
temperatura_miasta = temperatura[(temperatura["Lipiec"] == 20) |
                                   (temperatura["Styczeń"] == -1)]
print(temperatura_miasta)
```

a) Miasta gdzie temperatura w lipcu większa niż 19 stopni

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
1	Kraków	-1	10	20	9
3	Wrocław	-1	11	21	10
5	Łódź	-3	9	20	8

b) Miasta gdzie temperatura w styczniu większa lub równa -1

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
1	Kraków	-1	10	20	9
2	Gdańsk	0	8	17	10
3	Wrocław	-1	11	21	10

c) Miasta gdzie temperatura w kwietniu wynosi dokładnie 10 stopni

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
1	Kraków	-1	10	20	9
4	Poznań	-2	10	19	9

d) Miasta gdzie temperatura w lipcu jest 20 LUB temperatura w styczniu jest -1

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
1	Kraków	-1	10	20	9
3	Wrocław	-1	11	21	10
5	Łódź	-3	9	20	8

Zadanie 5: Złożone filtrowanie

a) Znajdź miasta, gdzie temperatura w lipcu > 18 ORAZ w styczniu >= -2

b) Znajdź miasta: 'Warszawa', 'Kraków', 'Gdańsk'

c) Znajdź miasta, gdzie temperatura w lipcu jest między 18 a 20 stopni (włącznie)

```
print("a) Miasta gdzie temperatura w lipcu jest 20 LUB temperatura w
styczniu jest -1")
temperatura_miasta = temperatura[(temperatura["Lipiec"] == 18) &
                                   (temperatura["Styczeń"] >= -2)]
print(temperatura_miasta)
```

```
print("\nb) Miasta: 'Warszawa', 'Kraków', 'Gdańsk'")
miasta_do_wyswietlenia = ["Warszawa", "Kraków", "Gdańsk"]
print(miasta_do_wyswietlenia)
```

```
print("\nc) Miasta gdzie temperatura w lipcu jest między 18 a 20
stopni (włącznie)")
print(temperatura[(temperatura["Lipiec"] >= 18) &
                  (temperatura["Lipiec"] <= 20)])
```

```
a) Miasta gdzie temperatura w lipcu jest 20 LUB temperatura w
styczniu jest -1
Empty DataFrame
Columns: [Miasto, Styczeń, Kwiecień, Lipiec, Październik]
Index: []
```

```
b) Miasta: 'Warszawa', 'Kraków', 'Gdańsk'
['Warszawa', 'Kraków', 'Gdańsk']
```

```
c) Miasta gdzie temperatura w lipcu jest między 18 a 20 stopni
(włącznie)
```

	Miasto	Styczeń	Kwiecień	Lipiec	Październik
0	Warszawa	-2	9	19	9
1	Kraków	-1	10	20	9

4	Poznań	-2	10	19	9
5	Łódź	-3	9	20	8

Zadanie 6: Dodawanie nowych kolumn

- Dodaj kolumnę 'Średnia_roczna' - średnia z wszystkich miesięcy
- Dodaj kolumnę 'Amplituda' - różnica między lipcem a styczniem
- Dodaj kolumnę 'Ciepłe_miasto' (True jeśli średnia roczna > 11, False w przeciwnym razie)

```
print("a) Stworzenie nowej kolumny 'Średnia_roczna'")
temperatura["Średnia_roczna"] = (temperatura["Styczeń"] +
                                temperatura["Kwiecień"] +
                                temperatura["Lipiec"] +
                                temperatura["Październik"]) / 4

print(temperatura)

print("\n b) Stworzenie nowej kolumny 'Amplituda'")
temperatura["Amplituda"] = temperatura["Lipiec"] -
temperatura["Styczeń"]
print(temperatura)

print("\n c) Stworzenie nowej kolumny 'Ciepłe_miasto'")
temperatura["Ciepłe_miasto"] = temperatura["Średnia_roczna"] > 11
print(temperatura)
```

a) Stworzenie nowej kolumny 'Średnia_roczna'

	Miasto	Styczeń	Kwiecień	Lipiec	Październik	Średnia_roczna
0	Warszawa	-2	9	19	9	8.75
1	Kraków	-1	10	20	9	9.50
2	Gdańsk	0	8	17	10	8.75
3	Wrocław	-1	11	21	10	10.25
4	Poznań	-2	10	19	9	9.00
5	Łódź	-3	9	20	8	8.50

b) Stworzenie nowej kolumny 'Amplituda'

	Miasto	Styczeń	Kwiecień	Lipiec	Październik	Średnia_roczna	Amplituda
0	Warszawa	-2	9	19	9	8.75	21
1	Kraków	-1	10	20	9	9.50	21
2	Gdańsk	0	8	17	10	8.75	17
3	Wrocław	-1	11	21	10	10.25	22
4	Poznań	-2	10	19	9	9.00	21
5	Łódź	-3	9	20	8	8.50	

23

c) Stworzenie nowej kolumny 'Ciepłe_miasto'

	Miasto	Styczeń	Kwiecień	Lipiec	Październik	Średnia_roczna \
0	Warszawa	-2	9	19	9	8.75
1	Kraków	-1	10	20	9	9.50
2	Gdańsk	0	8	17	10	8.75
3	Wrocław	-1	11	21	10	10.25
4	Poznań	-2	10	19	9	9.00
5	Łódź	-3	9	20	8	8.50

	Amplituda	Ciepłe_miasto
0	21	False
1	21	False
2	17	False
3	22	False
4	21	False
5	23	False

Zadanie: zaawansowane

```
# Nowy dataset dla zaawansowanego zadania
wyniki_egzaminy = pd.DataFrame({
    'Student': ['Anna', 'Bartek', 'Celina', 'Damian', 'Ewa', 'Filip',
                'Gabriela', 'Hubert'],
    'Matematyka': [85, 92, 78, 88, 95, 72, 89, 81],
    'Fizyka': [78, 88, 82, 75, 91, 68, 85, 79],
    'Informatyka': [92, 85, 88, 90, 89, 95, 82, 87],
    'Kierunek': ['Informatyka', 'Fizyka', 'Matematyka', 'Informatyka',
                 'Matematyka', 'Informatyka', 'Fizyka', 'Matematyka']
})
```

- Dodaj kolumnę 'Średnia' z wyników z trzech przedmiotów
- Dodaj kolumnę 'Status' - 'Zaliczony' jeśli średnia ≥ 85 , 'Niezaliczony' w przeciwnym razie
- Dla każdego kierunku znajdź studenta z najwyższą średnią:
 - Przefiltruj po kierunku
 - Posortuj według średniej malejąco
 - Weź pierwszy wiersz
 - Wykonaj dla każdego z 3 kierunków
- Znajdź wszystkich studentów informatyki, którzy mają średnią ≥ 85 i wynik z informatyki ≥ 90 , posortowanych według średniej malejąco

```
wyniki_egzaminy = pd.DataFrame({
    'Student': ['Anna', 'Bartek', 'Celina', 'Damian', 'Ewa', 'Filip',
                'Gabriela', 'Hubert'],
```

```

'Matematyka': [85, 92, 78, 88, 95, 72, 89, 81],
'Fizyka': [78, 88, 82, 75, 91, 68, 85, 79],
'Informatyka': [92, 85, 88, 90, 89, 95, 82, 87],
'Kierunek': ['Informatyka', 'Fizyka', 'Matematyka', 'Informatyka',
             'Matematyka', 'Informatyka', 'Fizyka', 'Matematyka']
})

student = pd.DataFrame(wyniki_egzaminu)

student.head()

{"summary": "{\n  \"name\": \"student\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"Student\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"Bartek\",\n          \"Filip\",\n          \"Anna\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Matematyka\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 7,\n        \"min\": 72,\n        \"max\": 95,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          92,\n          72,\n          85\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Fizyka\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 7,\n        \"min\": 68,\n        \"max\": 91,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          88,\n          68,\n          78\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Informatyka\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4,\n        \"min\": 82,\n        \"max\": 95,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          85,\n          95,\n          92\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Kierunek\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"Informatyka\",\n          \"Fizyka\",\n          \"Matematyka\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\",\n  \"variable_name\": \"student\"}

print("a) Średnia z wyników z trzech przedmiotów")
student["Średnia"] = (student["Matematyka"] + student["Informatyka"] +
student["Fizyka"]) / 3
print(student)

print("\n b) Status - 'Zaliczony' jeśli średnia >= 85, 'Niezaliczony'
w przeciwnym razie")
student["Status"] = student["Średnia"] >= 85
print(student)

```

```

print("\n c) Student każdego kierunku z najwyższą średnią:")
for kierunek in student["Kierunek"].unique():
    najlepszy_student = student[student["Kierunek"] ==
    kierunek].sort_values(
        by="Średnia", ascending=False
    ).iloc[0]
    print(f"\nNajlepszy student na kierunku {kierunek}:")
    print(najlepszy_student[["Student", "Średnia"]])

print("\n d) Studenci, którzy mają średnią >= 85 "
      "i wynik z informatyki >= 90, posortowani według średniej
malejąco"
)
studenci_informatyki_zaliczony = student[
    (student["Kierunek"] == "Informatyka")
    & (student["Średnia"] >= 85)
    & (student["Informatyka"] >= 90)
].sort_values(by="Średnia", ascending=False)
print(studenci_informatyki_zaliczony[["Student", "Średnia",
    "Kierunek", "Informatyka"]])

```

a) Średnia z wyników z trzech przedmiotów

	Student	Matematyka	Fizyka	Informatyka	Kierunek	Średnia
0	Anna	85	78	92	Informatyka	85.000000
1	Bartek	92	88	85	Fizyka	88.333333
2	Celina	78	82	88	Matematyka	82.666667
3	Damian	88	75	90	Informatyka	84.333333
4	Ewa	95	91	89	Matematyka	91.666667
5	Filip	72	68	95	Informatyka	78.333333
6	Gabriela	89	85	82	Fizyka	85.333333
7	Hubert	81	79	87	Matematyka	82.333333

b) Status - 'Zaliczony' jeśli średnia >= 85, 'Niezaliczony' w przeciwnym razie

	Student	Matematyka	Fizyka	Informatyka	Kierunek	Średnia	Status
0	Anna	85	78	92	Informatyka	85.000000	True
1	Bartek	92	88	85	Fizyka	88.333333	True
2	Celina	78	82	88	Matematyka	82.666667	False
3	Damian	88	75	90	Informatyka	84.333333	False
4	Ewa	95	91	89	Matematyka	91.666667	True
5	Filip	72	68	95	Informatyka	78.333333	False
6	Gabriela	89	85	82	Fizyka	85.333333	True

```
7    Hubert          81    79          87    Matematyka    82.333333
False
```

c) Student każdego kierunku z najwyższą średnią:

Najlepszy student na kierunku Informatyka:

```
Student    Anna
Średnia    85.0
Name: 0, dtype: object
```

Najlepszy student na kierunku Fizyka:

```
Student    Bartek
Średnia    88.333333
Name: 1, dtype: object
```

Najlepszy student na kierunku Matematyka:

```
Student    Ewa
Średnia    91.666667
Name: 4, dtype: object
```

d) Studenci, którzy mają średnią ≥ 85 i wynik z informatyki ≥ 90 ,
posortowani według średniej malejąco

```
Student  Średnia    Kierunek  Informatyka
0    Anna    85.0    Informatyka    92
```